



Particle Swarm Optimization Implementation and Application

Group member: Ziyu Ma, Jiayin Zhao
Student no.: 19-946-128, 22-912-505

Outline

- Basic Concept of PSO
- Extensions of PSO
- Test functions
- Comparison with `pyswarms.single.global_best` module
- Application
- Conclusion

Basic Concept

- Concept
- Mathematical Formula
- Pseudocode

Particle Swarm Optimization (PSO) Concept

- **Population Based Optimization:** use $M \gg N$ points simultaneously per iteration, individuals interact and profit from each other to find optimum together (inspired from nature: schools of fish, flock of birds, ant colonies...)
- Each particle has current location $\mathbf{p}_i(t)$ and velocity $\mathbf{v}_i(t)$, and they are updated in each iteration
- Each particle stores $\mathbf{p}_{i,best}$, $\mathbf{F}_{i,best}$ (individual's best fitness so far) and $\mathbf{p}_{i,best\ neighbor}$, $\mathbf{F}_{i,best\ neighbor}$ (group swarm's best fitness so far)
- Each iteration needs M fitness evaluations

Update Rules

$$\begin{aligned} v_i(t+1) = & \\ & w * v_i(t) \quad \text{inertia or momentum} \\ & + a_{ind} * R_{ind}(t) * (p_{i,best} - p_i(t)) \quad \text{to individual best (cognitive)} \\ & + a_{grp} * R_{grp}(t) * (p_{i,best\ neighbor} - p_i(t)) \quad \text{to group best (social)} \end{aligned}$$

$$p_i(t+1) = p_i(t) + v_i(t+1)$$

Free parameters:

- Dimension of function input (dim)
- Boundary of definition domain
- Swarm size M
- Velocity lower bound (minv) and upper bound (maxv)
- Number of maximum update iterations
- a_{ind} (cognitive term), a_{grp} (group component), w (inertia)
- Neighborhood topology
- Randomness through $R_{ind}(t)$ and $R_{grp}(t)$, uniform random number between [0,1]

Pseudocode of basic PSO (“full-mesh”)

Step1: Select hyperparameter values dim , boundary , M , minv , maxv , w , a_{ind} and a_{grp}

Step2: Randomly initialize swarm population of M particles X_i ($i=1, 2, \dots, M$) with velocity randomly within range $[\text{minv}, \text{maxv}]$, and position randomly within boundary

```
Step3: For iter in range(max_iter):           #loop max_iter times
      For i in range(M):                     #for each particle:
        a. Compute new velocity of ith particle
           swarm[i].velocity
           = w*swarm[i].velocity +
             r1*  $a_{\text{ind}}$  *(swarm[i].bestPos - swarm[i].position) +
             r2*  $a_{\text{grp}}$  *(best_pos_swarm - swarm[i].position)
        b. If velocity is not in range  $[\text{minv}, \text{maxv}]$  then clip it
           if swarm[i].velocity < minv:
             swarm[i].velocity = minv
           elif swarm[i].velocity > maxv:
             swarm[i].velocity = maxv
        c. Compute new position of ith particle using its new velocity
           swarm[i].position += swarm[i].velocity
           and make sure that position is not out of boundary
        d. Update new best of this particle and new best of Swarm
           if swarm[i].fitness < swarm[i].bestFitness:
             swarm[i].bestFitness = swarm[i].fitness
             swarm[i].bestPos = swarm[i].position
           if swarm[i].fitness < best_fitness_swarm:
             best_fitness_swarm = swarm[i].fitness
             best_pos_swarm = swarm[i].position
```

End-for

End-for

Step4: Return best particle of Swarm

Result of our implementation: basic PSO (“full-mesh”)

Here we use the Rastrigin function to show the result. To know more about the function, please go to slide 26. Results of other test functions will be shown later.

Parameters used: $w = 0.7$, $a_{ind} = a_{grp} = 1.5$

```
Goal is to minimize rastrigin's function in 2 variables  
Function has known min = 0 at (0, 0) Setting num_particles = 100  
Setting max_iter = 90
```

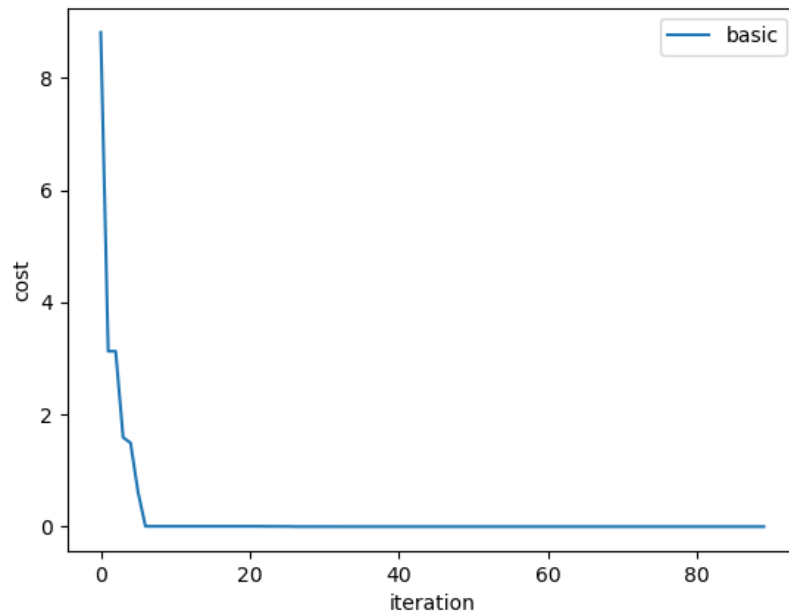
```
Starting PSO algorithm
```

```
Iter = 10 best fitness = 0.006  
Iter = 20 best fitness = 0.006  
Iter = 30 best fitness = 0.000  
Iter = 40 best fitness = 0.000  
Iter = 50 best fitness = 0.000  
Iter = 60 best fitness = 0.000  
Iter = 70 best fitness = 0.000  
Iter = 80 best fitness = 0.000
```

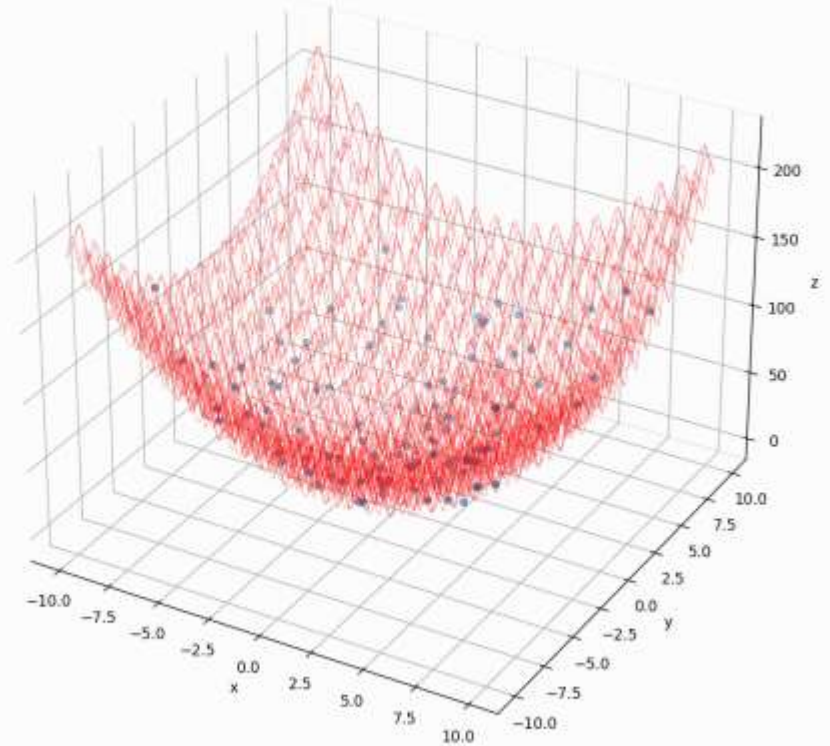
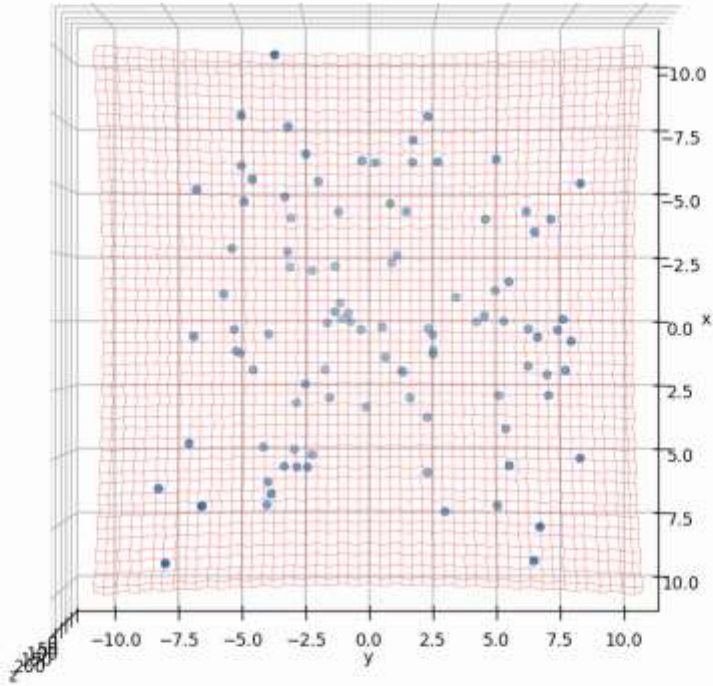
```
PSO completed
```

```
Best solution found:  
['-0.000000', '0.000000']  
fitness of best solution = 0.000000
```

```
End particle swarm for rastrigin function
```



Result of basic PSO (“full-mesh”)



Extensions

- Neighborhood topology
 - Pseudocode for extension
 - Comparison among the topologies
- Exploration and Exploitation groups
 - Pseudocode for extension
 - Demonstration of the effect using “full-mesh”
- From Static to Dynamic Hyperparameters
 - Pseudocode for extension
 - Demonstration of the effect

Neighborhood Topology



Neighborhood Topologies



Figure: Example population topologies. The leftmost picture depicts a fully connected topology, that is, $N_i = P \forall p_i \in P$ (self-links are not drawn for simplicity). The picture in the center depicts a so-called von Neumann topology, in which $|N_i| = 4 \forall p_i \in P$. The rightmost picture depicts a ring topology in which each particle is neighbor to two other particles.

[Particle swarm optimization - Scholarpedia](#)

Neighborhood Topologies

- Particles communicate with neighbors to get location $p_{i,best\ neigh}(t)$
- Particles feel forces towards $p_{i,best}$, $p_{i,best\ neigh}$. Forces increase with larger distance.

Pseudocode of PSO (“von Neumann” and “Ring”)

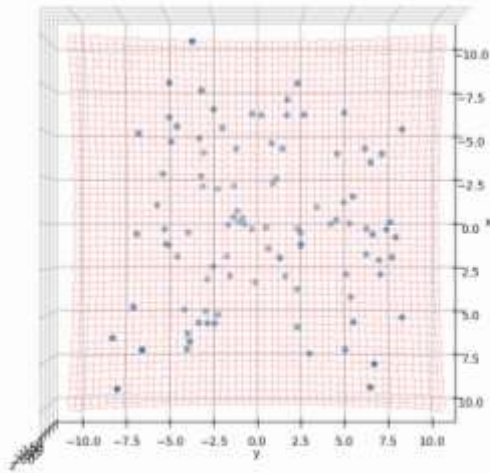
Step1: Select hyperparameter values dim , boundary , M , minv , maxv , w , a_{ind} and a_{grp} , **topology mode**

Step2: Randomly initialize swarm population of M particles X_i ($i=1, 2, \dots, M$) with velocity randomly within range $[\text{minv}, \text{maxv}]$, and position randomly within boundary

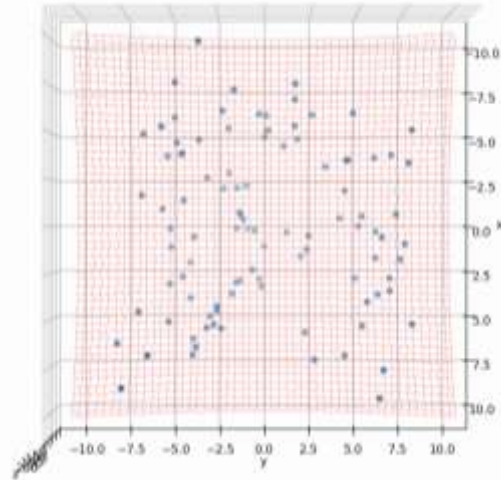
```
Step3: For iter in range(max_iter):           #loop max_iter times
    For i in range(M):                         #for each particle:
        a. Compute new velocity of ith particle
            best_pos_swarm_grp = best_pos_swarm
            if topology “Ring” or “von Neumann”:
                get neighbors of particle i according to the topology (2 if “Ring”, 4 if “von Neumann”).
                group G = neighbors U {i}
                best_pos_swarm_grp = swarm[j].bestPos for j among group G where swarm[j].bestFitness is the smallest
            swarm[i].velocity
                = w*swarm[i].velocity +
                  r1*  $a_{\text{ind}}$  *(swarm[i].bestPos - swarm[i].position) +
                  r2*  $a_{\text{grp}}$  *(best_pos_swarm_grp - swarm[i].position)
        b. If velocity is not in range  $[\text{minv}, \text{maxv}]$  then clip it
            if swarm[i].velocity < minv:
                swarm[i].velocity = minv
            elif swarm[i].velocity > maxv:
                swarm[i].velocity = maxv
        c. Compute new position of ith particle using its new velocity
            swarm[i].position += swarm[i].velocity
            and make sure that position is not out of boundary
        d. Update new best of this particle and new best of Swarm
            if swarm[i].fitness < swarm[i].bestFitness:
                swarm[i].bestFitness = swarm[i].fitness
                swarm[i].bestPos = swarm[i].position
            if swarm[i].fitness < best_fitness_swarm:
                best_fitness_swarm = swarm[i].fitness
                best_pos_swarm = swarm[i].position
    End-for
End-for
Step4: Return best particle of Swarm
```

Result of our implementation: “von Neumann” and “Ring”

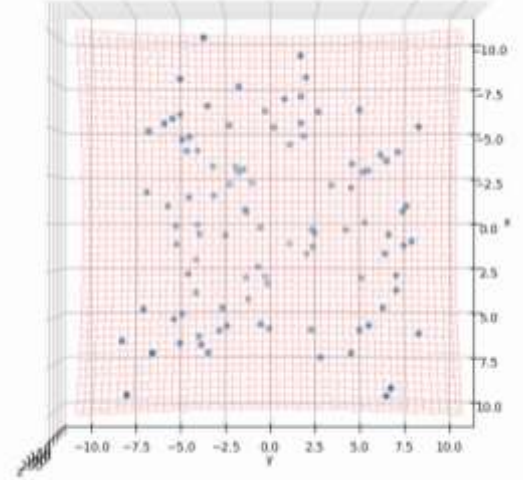
Basic



von Neumann



Ring

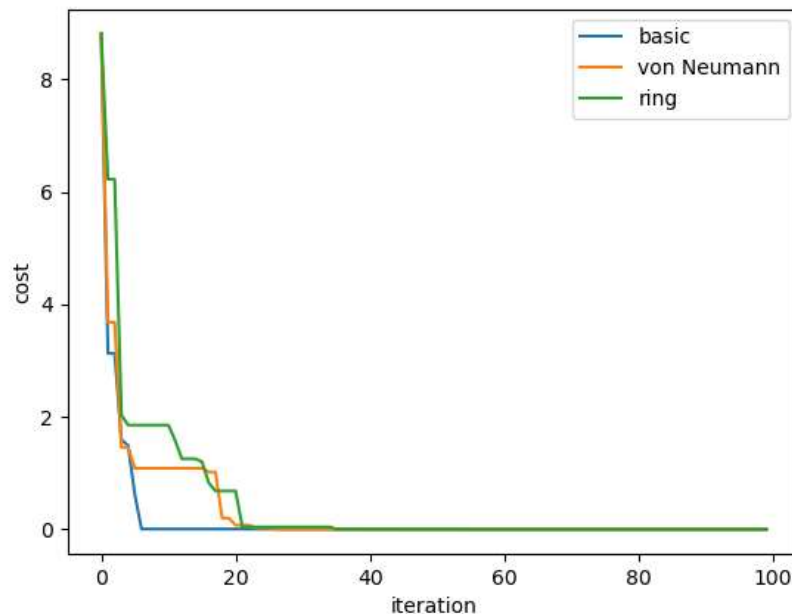


Parameters:

$$w = 0.7, a_{ind} = a_{grp} = 1.5$$

In the animation, we can see that particles converge to the (0,0,0) coordinate fastest with “Full-Mesh”, then with “von Neumann”, the slowest with “Ring”

Result of our implementation: “von Neumann” and “Ring”



With the same w , a_{ind} and a_{grp}

Information Exchange Speed

Full-Mesh > von Neumann > Ring

=> Speed of finding best solution in this case

Full-Mesh > von Neumann > Ring

Exploration and Exploitation groups



Exploration vs Exploitation

- **Exploration:** “explore different regions of the search space in order to locate a good optimum.” (Engelbrecht, 2007)
- **Exploitation:** “concentrate the search around a promising area in order to refine the candidate solution.” (Engelbrecht, 2007)
- Subdivide swarm in 2 groups
 1. **Monarch/Ruler:** responsible for **exploration**
focus on itself => large w , a_{ind} , and small a_{grp}
 2. **Worker:** responsible for **exploitation**
focus on the group => large a_{grp} and small w , a_{ind}
- Parameters:
 1. ratio of “Monarch-to-Worker” (e.g. 1/3 monarchs, and 2/3 workers)
 2. w , a_{ind} , a_{grp} for each group
(e.g. Monarch: $a_{ind} = 3$, $a_{grp} = 0.3$, and $w = 0.9$
Workers: $a_{ind} = 0.5$, $a_{grp} = 2$, and $w = 0.5$)

Pseudocode of PSO Extension (Monarch-Worker)

Step1: Select hyperparameter values dim , boundary , M , minv , maxv , w , a_{ind} and a_{grp} , **topology mode**, **workerRatio**, w_w , a_{ind_w} , a_{grp_w} (when **workerRatio** is 0, then no monarch-worker extension is used. Everything is like before, otherwise workers use w_w , a_{ind_w} , a_{grp_w} , and monarchs use w , a_{ind} and a_{grp})

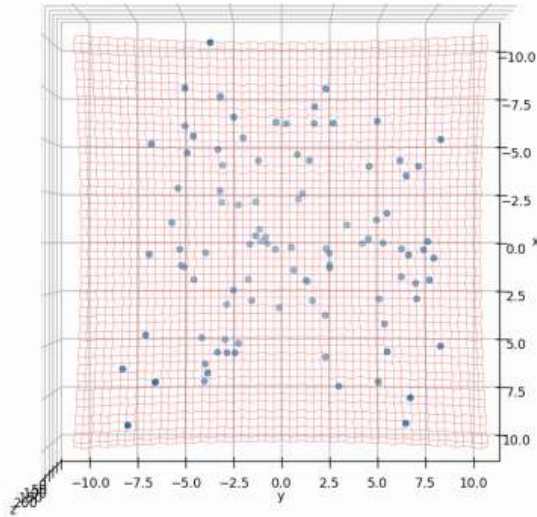
Step2: Randomly initialize swarm population of M particles X_i ($i=1, 2, \dots, M$) with velocity randomly within range $[\text{minv}, \text{maxv}]$, and position randomly within boundary
each particle has **workerRatio** possibility to be a worker, otherwise a monarch.

(We can also make the first **workerRatio** particles to workers, but it does not work well for “Ring” and “von Neumann” topologies since workers mainly communicate with workers and monarchs with monarchs. With this probabilistic, in expectation, we have $\text{workerRatio} * m$ workers.)

Step3: For iter in $\text{range}(\text{max_iter})$: #loop max_iter times
 For i in $\text{range}(M)$: #for each particle:
 a. Compute new velocity of i th particle
 $\text{best_pos_swarm_grp} = \text{best_pos_swarm}$
 if topology “Ring” or “von Neumann”:
 get neighbors of particle i according to the topology (2 if “Ring”, 4 if “von Neumann”).
 group $G = \text{neighbors } U \{i\}$
 $\text{best_pos_swarm_grp} = \text{swarm}[j].\text{bestPos}$ for j among group G where $\text{swarm}[j].\text{bestFitness}$ is the smallest
 if $\text{swarm}[i]$ is not a worker:
 $\text{swarm}[i].\text{velocity}$
 $= w * \text{swarm}[i].\text{velocity} +$
 $r1 * a_{ind} * (\text{swarm}[i].\text{bestPos} - \text{swarm}[i].\text{position}) +$
 $r2 * a_{grp} * (\text{best_pos_swarm_grp} - \text{swarm}[i].\text{position})$
 if $\text{swarm}[i]$ is a worker:
 $\text{swarm}[i].\text{velocity}$
 $= w_w * \text{swarm}[i].\text{velocity} +$
 $r1 * a_{ind_w} * (\text{swarm}[i].\text{bestPos} - \text{swarm}[i].\text{position}) +$
 $r2 * a_{grp_w} * (\text{best_pos_swarm_grp} - \text{swarm}[i].\text{position})$
 b. ... like before

Result of our implementation: monarch-worker with “full-mesh”

Basic



Parameters:

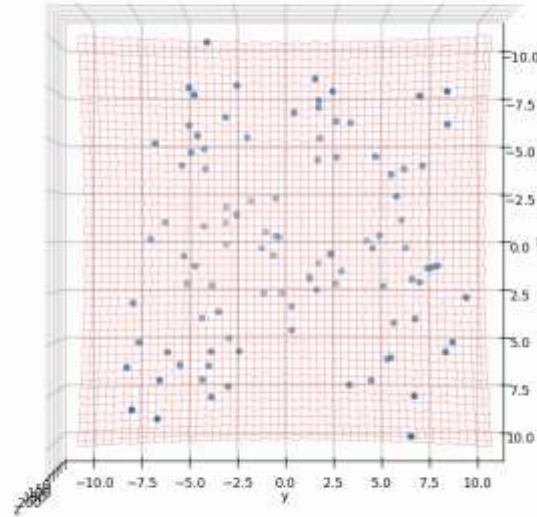
Monarch:

$w = 0.9$, $a_{ind} = 3$, $a_{grp} = 0.3$

Worker (33% of population):

$w = 0.5$, $a_{ind} = 0.5$, $a_{grp} = 2$

Monarch-Worker



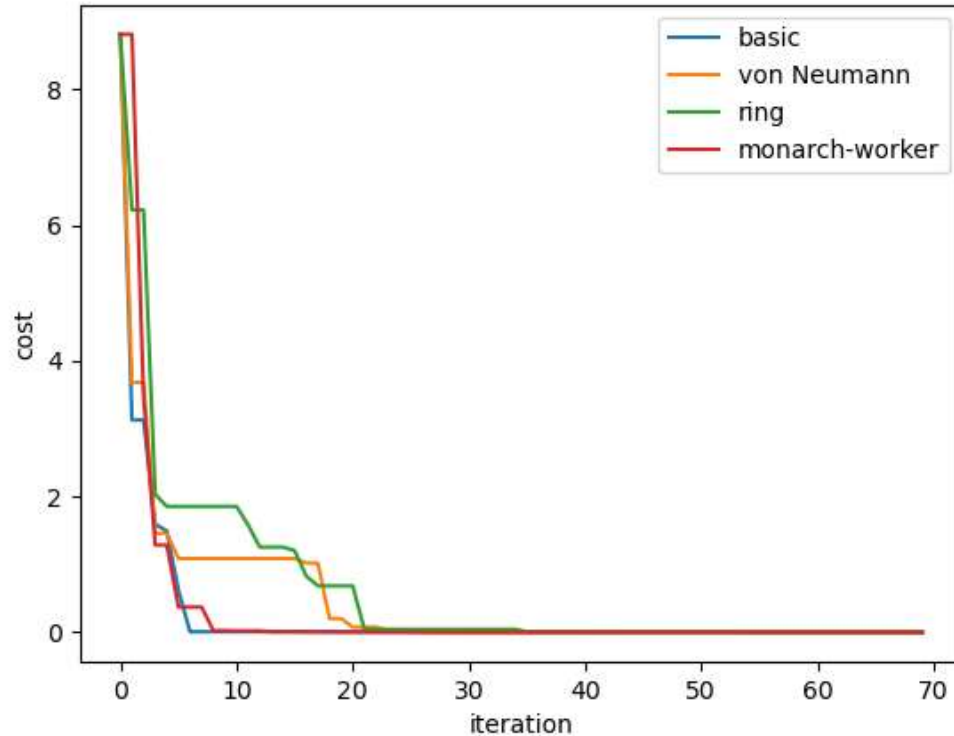
Without Monarch-Worker:

- all particles move toward the optimum

With Monarch-Worker:

- part of them move toward the optimum and stay there to exploit, part of them explores the rest of the space

Result of our implementation: monarch-worker with “full-mesh”



From Static to Dynamic Hyperparameters



Static vs Dynamic Hyperparameters

- **Static:** what we done before, assign to each particle the same w , a_{ind} , a_{grp} through the entire lifetime
- **Dynamic:** linear change of the hyperparameter over the number of iterations (Ratnaweera, 2004).
 - $w = (w^{max} - w^{min}) * \frac{i^{max} - i}{i^{max}} + w^{min}$
 - $a_{ind} = (a_{ind}^{min} - a_{ind}^{max}) * \frac{i}{i^{max}} + a_{ind}^{max}$
 - $a_{grp} = (a_{grp}^{max} - a_{grp}^{min}) * \frac{i}{i^{max}} + a_{grp}^{min}$
- The closer to i^{max} , the particles focuses less on itself and more on the best group result. It is reasonable since the more rounds we are been through, the more probable that a good result is found by one of our group member.
- Parameters:
 - Used before:
 - i^{max} : max iteration
 - i : the current round we are in
 - New for dynamic parameterization:
 - $[w^{min}, w^{max}]$ (e.g. [0.2, 0.9])
 - $[a_{ind}^{min}, a_{ind}^{max}]$ (e.g. [0.3, 3])
 - $[a_{grp}^{min}, a_{grp}^{max}]$ (e.g. [0.3, 3])

Pseudocode of PSO Extension (Dynamic Hyperparameters)

Step1: Select hyperparameter values dim, boundary, M, minv, maxv, w, a_{ind} and a_{grp} , topology mode, workerRatio, w_w, a_{ind_w} , a_{grp_w} , dynamic, wmin, wmax, aindmin, aindmax, agrpmin, agrpmax

Step2: Randomly initialize swarm population of M particles X_i ($i=1, 2, \dots, M$) with velocity randomly within range [minv, maxv], and position randomly within boundary
each particle has workerRatio possibility to be a worker, otherwise a monarch.

Step3: For iter in range(max_iter): #loop max_iter times
 if dynamic == 1:
 set w, aind, agrp accordingly
 if workerRatio != 0:
 also set w_w, aind_w, agrp_w the same way (by dynamic hyperparameters we use no monarch-worker extension)
 For i in range(M): #for each particle:
 a. Compute new velocity of ith particle
 best_pos_swarm_grp = best_pos_swarm
 if topology "Ring" or "von Neumann":
 get neighbors of particle i according to the topology (2 if "Ring", 4 if "von Neumann").
 group G = neighbors U {i}
 best_pos_swarm_grp = swarm[j].bestPos for j among group G where swarm[j].bestFitness is the smallest
 if swarm[i] is not a worker:
 swarm[i].velocity
 = w*swarm[i].velocity +
 $r1 * a_{ind} * (\text{swarm}[i].\text{bestPos} - \text{swarm}[i].\text{position}) +$
 $r2 * a_{grp} * (\text{best_pos_swarm_grp} - \text{swarm}[i].\text{position})$
 if swarm[i] is a worker:
 swarm[i].velocity
 = w_w*swarm[i].velocity +
 $r1 * a_{ind_w} * (\text{swarm}[i].\text{bestPos} - \text{swarm}[i].\text{position}) +$
 $r2 * a_{grp_w} * (\text{best_pos_swarm_grp} - \text{swarm}[i].\text{position})$
 b. ... like before

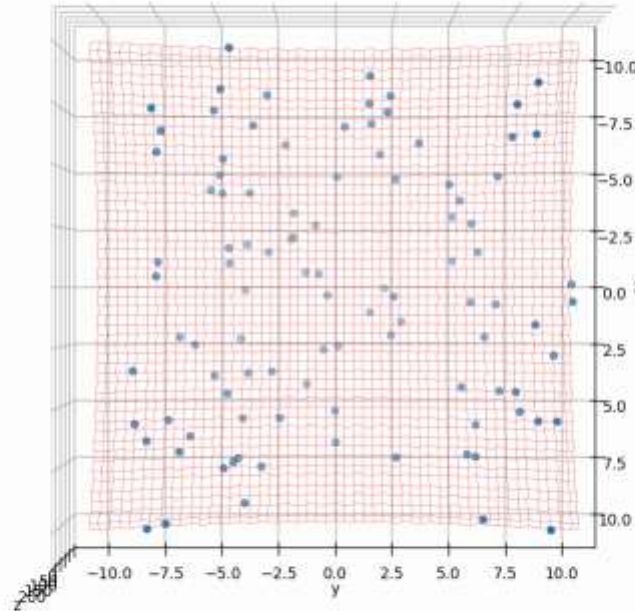
Result of our implementation: dynamic hyperparameters

Parameters used:

$$[w^{min}, w^{max}] = [0.1, 0.9]$$

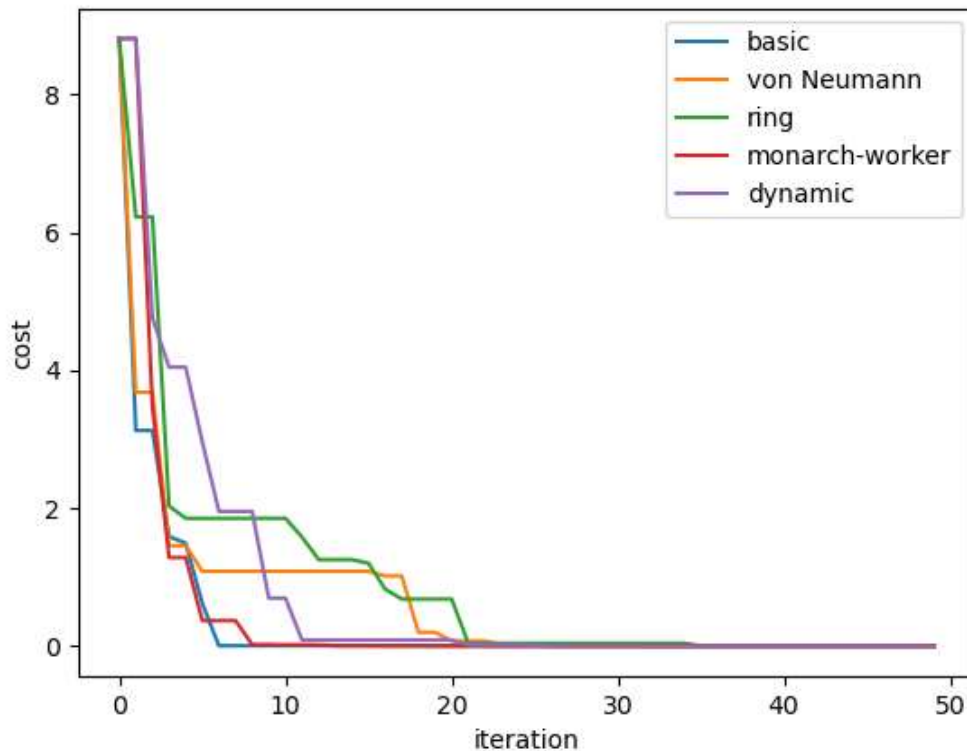
$$[a_{ind}^{min}, a_{ind}^{max}] = [0.1, 3]$$

$$[a_{grp}^{min}, a_{grp}^{max}] = [0.1, 3]$$



We found that these particles are initially distributed in the entire area, and as the value of round increases, they quickly orient to the optimal solution

Result of our implementation: dynamic hyperparameters



Test functions



Test functions

1. Rastrigin function:

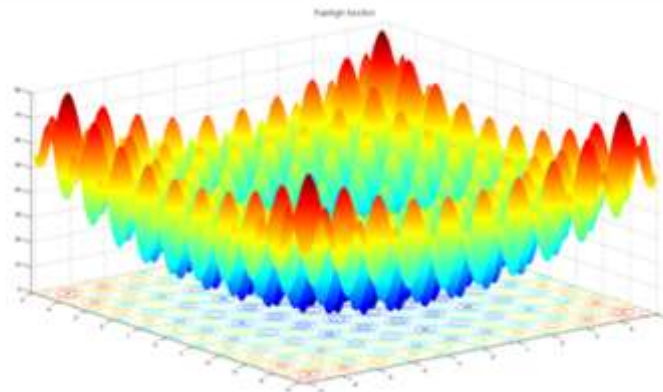
Rastrigin function is a non-convex function and is often used as a performance test problem for optimization algorithms.

function equation:

$$f(x_1 \cdots x_n) = 10n + \sum_{i=1}^n (x_i^2 - 10\cos(2\pi x_i))$$

minimum at $f(0, \dots, 0) = 0$

For an optimization algorithm, rastrigin function is a very challenging one. Its complex behavior cause optimization algorithms to often stuck at local minima. Having a lot of cosine oscillations on the plane introduces the complex behavior to this function.



Result

Demos are shown in the previous slides, global optimum found by our implementation!

Test functions

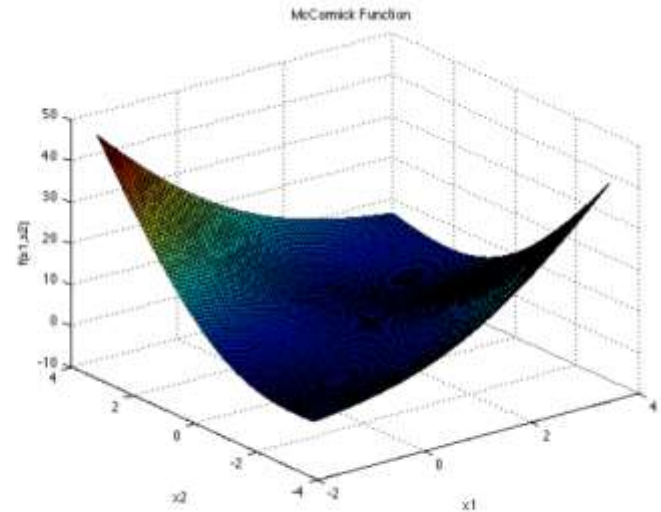
2. McCormick function

Input Domain:

The function is usually evaluated on the rectangle $x_1 \in [-1.5, 4]$, $x_2 \in [-3, 4]$.

Global Minimum:

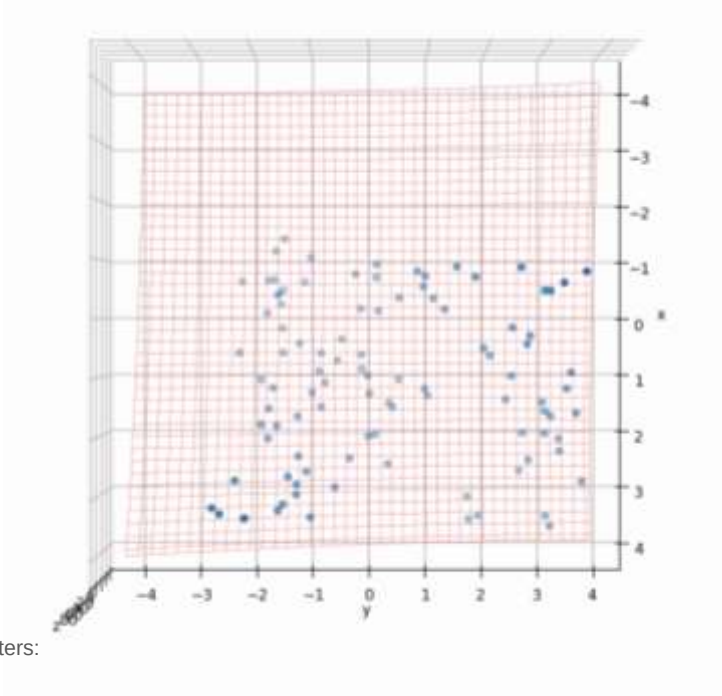
$f(\mathbf{x}^*) = -1.9133$, at $\mathbf{x}^* = (-0.54719, -1.54719)$



$$f(\mathbf{x}) = \sin(x_1 + x_2) + (x_1 - x_2)^2 - 1.5x_1 + 2.5x_2 + 1$$

[McCormick Function \(sfu.ca\)](http://sfu.ca)

Result



Parameters:

Basic version with $w = 0.5$, $a_{ind} = 1$, $a_{grp} = 1.5$ and "Full Mesh"

Boundary = $[-1.5, 4]$ for x and $[-3, 4]$ for y axis

min and max velocity about 5% of the domain range

```
Goal is to minimize McCormick's function in 2 variables
Function has known min = -1.9133 at (-0.54719, -1.54719)
Setting num_particles = 100
Setting max_iter = 50
```

```
Starting PSO algorithm
```

```
Iter = 10 best fitness = -1.913
Iter = 20 best fitness = -1.913
Iter = 30 best fitness = -1.913
Iter = 40 best fitness = -1.913
```

```
PSO completed
```

```
Best solution found:
['-0.547198', '-1.547198']
fitness of best solution = -1.913223
```

```
End particle swarm for McCormick function
```

Global optimum found by our implementation!

Result

We also tried other possibilities and compared them.

Parameters:

Basic: $w = 0.5$, $a_{ind} = 1$, $a_{grp} = 1.5$, “Full Mesh”

Monarch-worker:

monarch ($w = 0.8$, $a_{ind} = 2$, $a_{grp} = 0.3$)

worker (33% of population, $w = 0.3$, $a_{ind} = 0.5$, $a_{grp} = 2$)

Von Neumann: $w = 0.5$, $a_{ind} = 1$, $a_{grp} = 1.5$, “von Neumann”

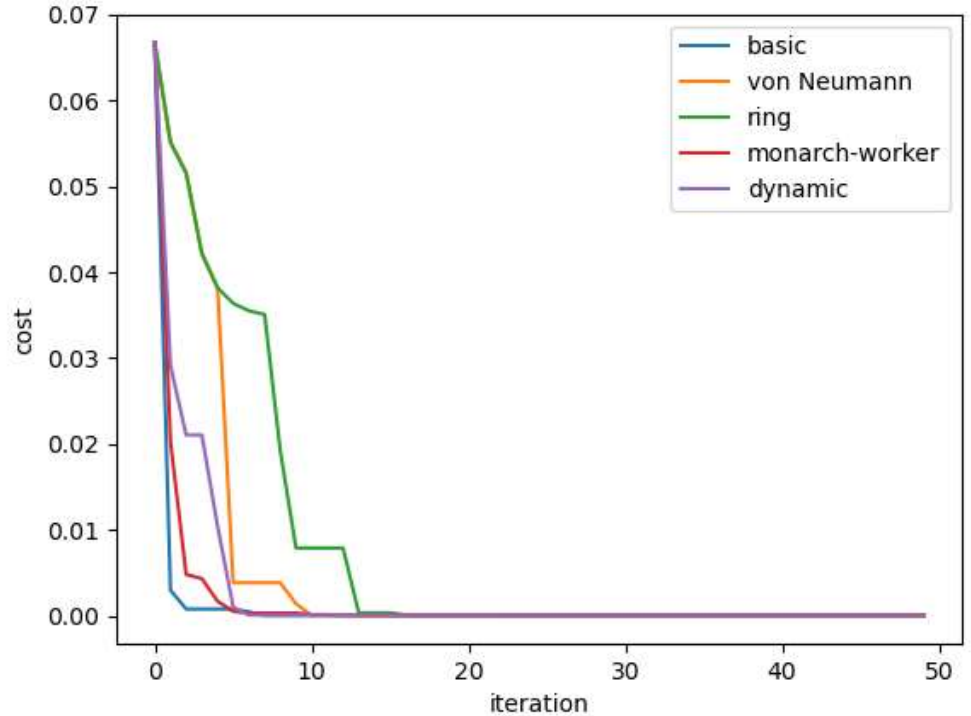
Ring: $w = 0.5$, $a_{ind} = 1$, $a_{grp} = 1.5$, “Ring”

Dynamic:

w from $[0.1, 0.8]$

a_{ind} from $[0.1, 2]$

a_{grp} from $[0.1, 3]$



Test functions

2. Eggholder function

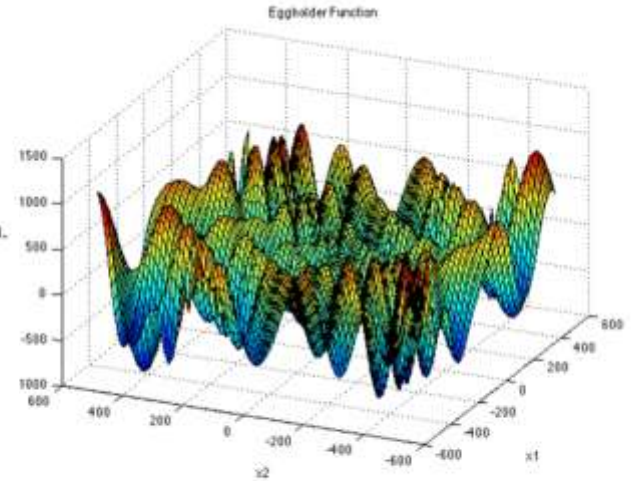
The Eggholder function is a difficult function to optimize, because of the large number of local minima.

Input Domain:

The function is usually evaluated on the square $x_i \in [-512, 512]$, for all $i = 1, 2$.

Global Minimum:

$f(\mathbf{x}^*) = -959.6407$, at $\mathbf{x}^* = (512, 404.2319)$



$$f(\mathbf{x}) = -(x_2 + 47) \sin \left(\sqrt{\left| x_2 + \frac{x_1}{2} + 47 \right|} \right) - x_1 \sin \left(\sqrt{|x_1 - (x_2 + 47)|} \right)$$

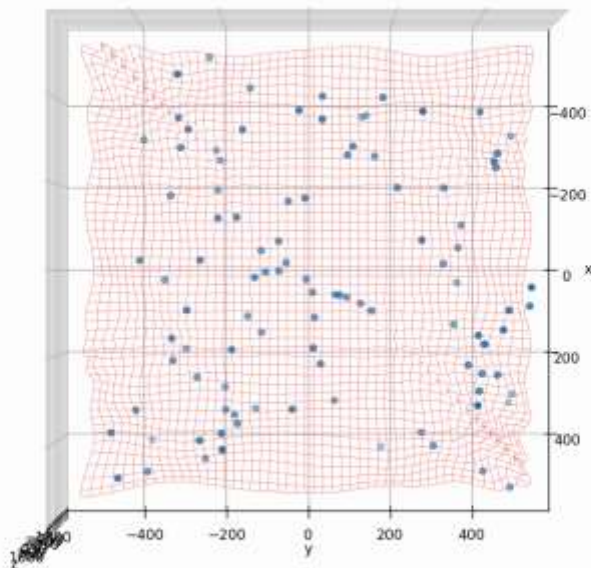
[Eggholder Function \(sfu.ca\)](http://sfu.ca)

Result

Basic version with $w = 0.9$, $a_{ind} = 3$, $a_{grp} = 0.3$ and “Full Mesh”

Boundary = $[-512, 512]$ for x and y axis

min and max velocity about 5% of the domain range



```
Goal is to minimize Eggholder's function in 2 variables
Function has known min = -959.6407 at (512, 404.2319) Setting num_particles = 100
Setting max_iter = 140
```

Starting PSO algorithm

```
Iter = 10 best fitness = -919.658
Iter = 20 best fitness = -950.459
Iter = 30 best fitness = -950.459
Iter = 40 best fitness = -957.140
Iter = 50 best fitness = -957.140
Iter = 60 best fitness = -958.802
Iter = 70 best fitness = -959.133
Iter = 80 best fitness = -959.613
Iter = 90 best fitness = -959.613
Iter = 100 best fitness = -959.634
Iter = 110 best fitness = -959.634
Iter = 120 best fitness = -959.634
Iter = 130 best fitness = -959.640
```

PSO completed

```
Best solution found:
['512.000000', '404.214142']
fitness of best solution = -959.640308
```

End particle swarm for eggholder function

Global optimum found by our implementation!

Result

We also tried other possibilities and compared them.

Parameters:

Basic: $w = 0.9$, $a_{ind} = 3$, $a_{grp} = 0.3$, velocity limit 5% of definition domain, “Full Mesh”.

Monarch-worker:

monarch ($w = 0.9$, $a_{ind} = 3$, $a_{grp} = 0.3$)

worker (20% of population, $w = 0.1$, $a_{ind} = 0.1$, $a_{grp} = 2$),

velocity limit 5% of definition domain.

Von Neumann: $w = 0.648$, $a_{ind} = 4$, $a_{grp} = 0.389$, “von Neumann”, velocity limit 2.5% of definition domain.

Ring: $w = 0.759$, $a_{ind} = 4.1$, $a_{grp} = 0.32$, “Ring”, velocity limit 2.5% of definition domain.

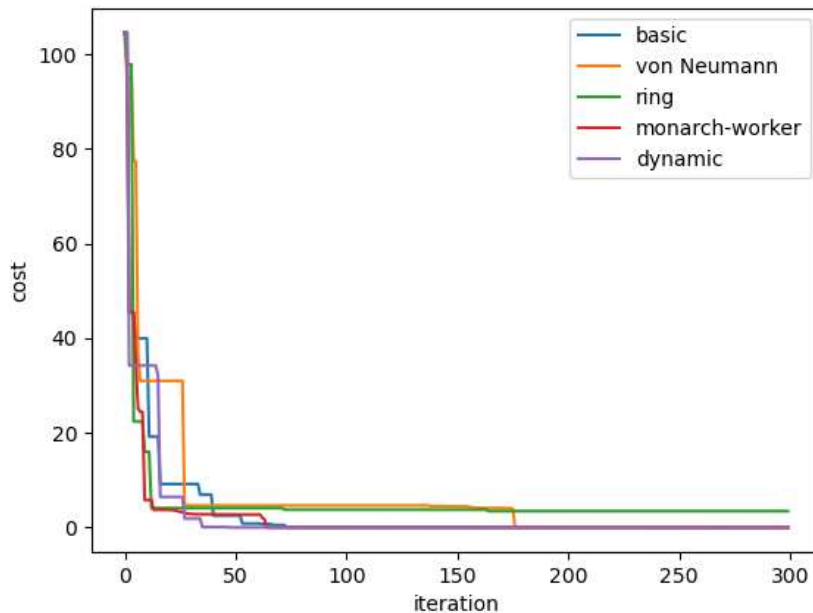
Dynamic:

w from $[0.1, 0.9]$

a_{ind} from $[0.1, 3]$

a_{grp} from $[0.3, 2]$

velocity limit 5% of definition domain.



We still think there is a combination of parameters for “ring” to reach the minimum.

Comparison with pyswarm

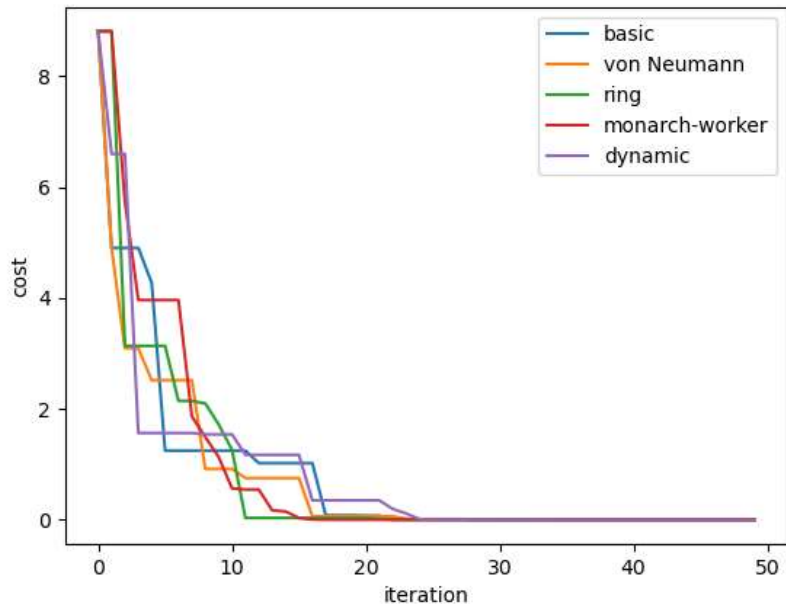


Finding the minimum

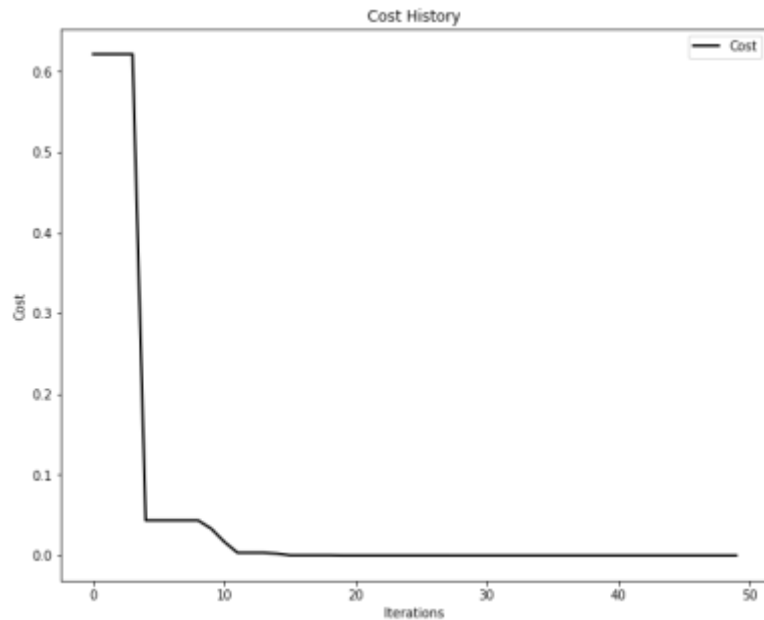


Rastrigin

Implemented by us



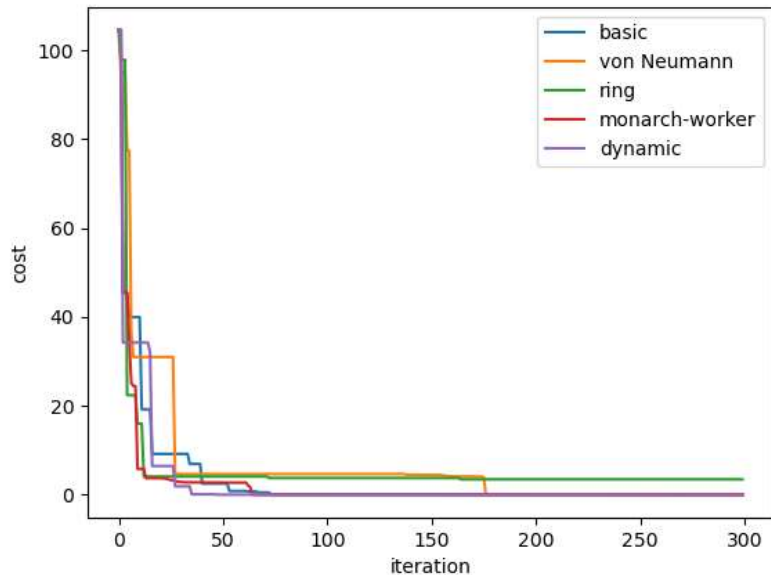
`pyswarms.single.global_best` module
(same population and iteration as our implementation)
 $c1 = c2 = 0.5$, $w = 0.7$



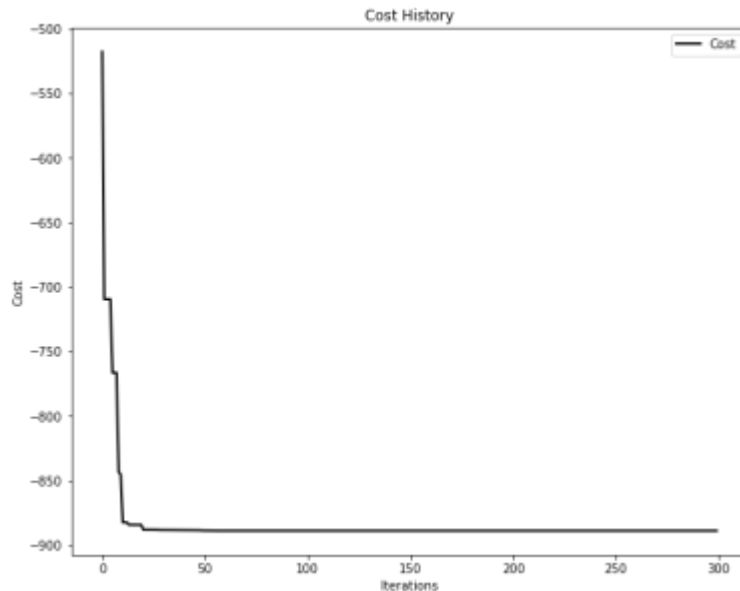
Pyswarm is faster at founding the minimum.

Eggholder

Implemented by us



`pyswarms.single.global_best` module
(same population and iteration as our implementation)
 $c1 = c2 = 0.5, w = 0.5$



Pyswarm is quite similar with ours to find the minimum.

CPU time for chosen rounds



Rastrigin (execution time for 100 rounds)

Pyswarm	140ms
Basic	182ms
Von Neumann	217ms
Ring	196ms
Monarch-worker	183ms
Dynamic	182ms

Pyswarm is faster. Among the versions that we implemented, “von Neumann” and “Ring” are expected to be slower since we need to explicitly keep track of the distinct neighbors for each particle.

Eggholder (execution time for 150 rounds)

Pyswarm	270ms
Basic	261ms
Von Neumann	293ms
Ring	315ms
Monarch-worker	296ms
Dynamic	270ms

Quite similar!

Application



Optimization weight for Regression model by PSO

Using PSO to search in vector space, aim to implement it as well as use it to search the vector space of the mean squared error function and tried to minimize it and fit the model.

```
training loss 24.45429987823288 of epoch 47  
training loss 24.432607125180482 of epoch 48  
training loss 24.38212660051493 of epoch 49  
training loss 24.356004051389423 of epoch 50  
(14,)  
(50, 13)  
10.53505047731465
```

```
from sklearn.linear_model import LinearRegression  
mod = LinearRegression()  
mod.fit(X[:-50], y[:-50])  
print(model.loss_func(y[-50:], mod.predict(X[-50:])))  
10.960410679422752
```

Find that this was at par or sometimes even better than the standard Normal equation.

Conclusion



Advantages of PSO

- Derivative free.
- Very efficient global search algorithm.
- Easily parallelized for concurrent processing.

Disadvantages of PSO

- Efficiency and competitiveness questionable (many fitness evaluations, no conceptual simplicity).

Summary of what we have done

- We implemented
 - Basic (“full-mesh”) PSO
 - “von Neumann” topology
 - “Ring” topology
 - Monarch-Worker extension
 - Dynamic hyperparameters
- We played around with different combinations of input parameters.
- We tested the implementation using the Rastrigin, McCormick and Eggholder functions. Almost all versions of our implementations found all the functions’ global optima.
- We compared our implementation with **pyswarms.single.global_best module**.

More considerations

- It is hard to say which version (“full-mesh”, “ring”, “von Neumann”, monarch-worker, dynamic) is the best. They all have the potential to perform best in certain application scenarios.
- But it seems that “full-mesh” and “monarch-worker” are good choices.
- No combination of w , a_{ind} and a_{grp} is always the best. We need to choose w , a_{ind} and a_{grp} carefully according to our fitness function. For functions with many local optimums (e.g. eggholder), bigger w and a_{ind} might be helpful. For a smooth function with one obvious global optimum (e.g. McCormick), bigger a_{grp} might be useful.
- The result is relatively sensitive to the parameters that we use, also the min and max velocities.
- It is hard to say if our implementation outperforms pyswarm, since it depends on the parameters that we use and the functions that we test.

Thanks for reading!